Final Next-Step Package: Payment-Control Review

Oct 15, 2026 · @Shantha

0. Status, boundary and how to read this package

Prepared under your instruction of 15 October from the consolidated plan, the five reports and the completed evidence, with static code findings already on record. Nothing was executed: no SQL, no fix, no change to data, schema or permissions, no staff assignment, no file written to your computer, no transfer, no outside workspace or external script used, nothing installed; live, avenir_rv and the cloud test database were not accessed. **[Observed]**

Standing assumptions you set, applied throughout. The historical data is treated as potentially inaccurate and the inherited code as potentially faulty: existing behaviour is not taken as correct behaviour. Your reported observations (paid items reappearing in the payment panel; invoices shown two or three times) are treated as reports whose causes are not yet established. Ravi is the accountant; staff and access decisions are yours. The April 1 rebuild is noted as an intention only: no date is assumed, no record reset, nothing imported.

Labels. [Observed] = read from evidence or code already on record, with its source. [Deduced] = follows from observed facts. [Proposal] = for your decision. [Assumption] = not verified; stated so you can see it. Approved evidence location throughout: C:\wamp64\www\avenir\writable\AvenirEvidence\2026-10-14-stage1 (your instruction of 15 October; recognised, not re-asked).

Where to look first. Section 11 lists the approvals, grouped; D8 is optional and blocks nothing. Section 5 holds the exact statements and checksums. Section 7 answers the recurring-payment question as far as the existing findings allow, and names the one blocker that needs a word from you, not an investigation.

1. Amendments to the consolidated plan

#	Plan section	Amendment	Why
A1	3c, capture C1	S1 (secure_file_priv) is no longer treated as sufficient. Three prerequisites are verified in order: S1 the setting, S2 the FILE privilege of the signed-in account, S3 a one-row probe write into followup-run\ (section 3). Only after S3 succeeds is a result statement run with INTO OUTFILE	your point 2: an empty secure_file_priv alone does not guarantee a write
A2	3c, 5	The INTO OUTFILE texts are issued now, with the approved folder path, each with its own SHA-256 (section 5), so approval can be given once; the plain Revision 1 checksums remain the reference for the SELECT part	capture requires new checksums; issuing them now avoids a second approval round
A3	3c	What C1 preserves and what it does not is stated explicitly: rows by MySQL into the folder; errors, Notes and timings only in phpMyAdmin's response, which C1 does not write; a run log inside the folder is proposed for them (section 3)	your point 2
A4	4c, 5	BK1 attribution is restated: ledger table plus journal id, with the entry timestamp Dt as the tie-breaker for reused ids; rows under Recpt_No 7145 are candidates only, because 7145 is also a voucher number in its own right (section 4)	your point 3; M2 shows ID is not unique in either ledger table

#	Plan section	Amendment	Why
A5	7, order	BK1 stays before R20.2r; S1–S3 and M3 precede both; H1 is read before and after each capped statement as Revision 1 requires, so four H1 files under C1	unchanged in substance; made explicit
A6	7, decisions	D1–D8 regrouped into section 11 here; D8 marked optional and non-blocking; two new approvals added (S2, S3) and one removed (no separate approval for the C1 texts beyond their checksums)	your point on approvals
A7	new	Sections 7–9 added: the recurring-payment flow map, the four kinds of duplicate, focused test cases (not executed) and a prioritised repair-and-test proposal that keeps software faults apart from historical-data reconciliation	your point 4
A8	5 (plan)	The Ravi pack gains the rule that suspected duplicates are kept identifiable with their references and reasons, and that no record is deleted, merged, reversed or cleaned up by me (section 10)	your point 5
A9	3a	The missing text of the structural restriction remains missing; nothing in this package depends on my reading of it beyond your 12:01 UTC instruction and the present one, which are applied as written	unchanged

Everything else in the consolidated plan stands: the 21 completed statements and the code checks are not repeated (section 2 says when a repeat would become necessary); Revision 1 governs the remaining checks; the evidence location is approved.

2. What the existing evidence establishes, and what remains unknown

Established [Observed, with source]

Finding	Source
The target is localhost MySQL 8.4.7, port 3306, schema avenir, signed in as root@localhost; timeout settings 0; performance_schema on	T1, H0 (15 Oct)
MySQL applied the 60-second cap to all eight capped statements and none ran over; E1′s EXPLAIN also counts as a set timeout	H1 × 10
Ledger tables: MyISAM, no primary key, ID non-unique; voucher has a unique index on Vnum; v_inv has no index on V_tkn; Date is date, Dr/Cr decimal(19,2)	M1, M2
Voucher 7134 carries two identical voucher-payment journals (26061 at 15:24:03, login 75; 26075 at 16:28:48, login 1, both 13 Jul 2026), each Dr 6-9-23-11013 / Cr 1-6-8-9750 for 298,375.00; every other sup = 0 authorised voucher carries one. This is the whole LKR 298,375.00 difference between R0′s ledger figure and R23.1′s voucher total	G1c, G2r, G3, G4
Under the 38 sup = 0 numbers no journal debits 1-6-8-9750; all 9750 credits are “voucher payment” rows; none carries Invoice_Num ′0′; no rows in the 2024 table; no shared voucher numbers	G2r
aflag holds only 0 and 3; no row has aflag 1; Bank_Date is never set; the category imbalances in account_runing equal the eight R21 journals; the 41 Stage One rows stand	L1, L2
E1′s plan: UNION sets materialised once, index lookups into them, Recpt_No index antijoins, no ledger scan in the voucher loop; two Notes #1276, no Warning, no Error	E1

Finding	Source
The code has four authorisation paths; only the desk refuses a second authorisation; at least eleven ways exist to pay or post one obligation twice (D1–D11); CSRF is not checked; request approval, rejection, cancellation and completion routes have no status checks or locks except where noted	follow-up report section 6; A2 route inventory; R0
payment_request statuses on development on 14 Oct: 0: 2, 1: 10, 2: 2,159, 3: 8, 4: 891, 5: 25, 6: 15; 266 requests at status 4 show the “paid” flag 1, 21 at status 2 show flag 0; bulk rejection sets a bulk-type request to 6	R0
The 36 finalized sup = 0 bank transfers name 1-6-8-10380 as their bank, but no credit to that ledger was found for them	Stage One section 7; R0

Not established [Unknown]

Question	Why it is open	What would answer it
Whether the bank paid 298,375.00 once or twice for voucher 7134. Your clarification of 15 October (section 12): the monthly bank checks found no duplicate bank payment; the finding is classified as a duplicate ledger posting. Ravi confirms the correction against the bank reconciliation	the duplicate is in the postings; no read so far touches the bank statement, and no bank-side journal is filed under the voucher's number	Ravi against the Commercial Bank statement (section 10); BK1 only narrows the ledger side
Why journal 26075 was posted 64 minutes after 26061, by a different login	no audit before 11 Oct; the route V1 has no Aflag guard, so a second authorisation posts again, but the sequence of events is not recorded	not answerable from data; the guard A1 prevents recurrence
Which journal Ravi will treat as the duplicate	an accounting decision	Ravi

Question	Why it is open	What would answer it
Whether any further bank-side or clearing rows exist for 7134 under another number	not read	BK1, with the attribution rules of section 4
How many of the 3,921 vouchers not found by number have a same-date, same-amount candidate	R20.2′s result was never captured	R20.2r
What “paid items reappear in the panel” and “invoices appear two or three times” refer to on the live system: which screen, which column, which records	reports only; the live system is out of scope and the screen is not named	one question to you (section 7); then a read of the named screen′s query on localhost, under approval
Whether the inherited code′s panel queries select by request status, by tour flags, or both	the flag-based dashboards are documented (r3.3 section 5b; R0 R16), but not every panel′s query has been read	a static read of the named screen′s controller, under approval
Whether Dt, Cashier and Invoice_Num exist on account_runing_2024	M1 did not list them	M3
Whether secure_file_priv, the FILE privilege and folder permissions allow MySQL to write into followup-run\	not read	S1, S2, S3
The text of the standing structural restriction	not in this session′s record	you

When a completed check would need repeating. T1/H0 if the server, port, schema or account differs at the next sign-in; the H1 baseline immediately before any new capped statement (always); F-5 and the code reviews if any reviewed file′s SHA-256 changes; M1/M2 if a schema change is installed. None of these conditions is known to hold now.

3. Capture details resolved: prerequisites verified, what is preserved and how

Method C1, as in the plan: each approved statement is sent through the signed-in phpMyAdmin page exactly as on 15 October, with one added final clause, `INTO OUTFILE '<approved folder>/followup-run/NN_ID.tsv'`. MySQL itself writes the rows into the approved folder on your computer. No outside workspace, no transfer, no external script, no installation, no change to server settings or privileges. Every such statement is a new text with its own SHA-256 (section 5). **[Proposal]**

3a. Prerequisites, verified in order rather than assumed

Step	Verifies	Pass	Fail →
S1	`secure_file_priv`: '' (empty) permits any folder; a path permits only that folder; NULL disables file output	'' or a path that is, or contains, the approved folder	C1 unavailable without a server-setting change, which is not proposed; use C2 (your phpMyAdmin export into the folder)
S2	the signed-in account holds the FILE privilege (information_schema.USER_PRIVILEGES for 'root'@'localhost')	one row, PRIVILEGE_TYPE FILE	C1 unavailable without a grant, which is not proposed; use C2
S3	the MySQL service can create a file in followup-run: a one-row probe `SELECT 1 AS probe INTO OUTFILE '…/24_S3_probe.tsv'`	phpMyAdmin reports 1 row affected and the file exists with content `1`	the folder's permissions for the MySQL service account are IT's to adjust; use C2 meanwhile

S3 is itself the first write into the folder, so it carries its own approval line (section 11). A file that already exists makes MySQL refuse the write (error 1086), so every file name is used once; a repeat run needs a new name and a new checksum.

3b. What C1 preserves inside the folder, and what it does not

Evidence	Under C1	How it is preserved
Result rows	**yes** — written by MySQL to NN_ID.tsv: tab-separated, one line per row, NULL as `\N`, backslash-escaped, no header line	the header is the statement's own column aliases; the file is paired with its statement text in statements\capture-c1\ and its SHA-256 recorded in SHA256SUMS-2
Row count cross-check	yes	phpMyAdmin's "N rows affected" must equal the file's line count; a difference is a stop
SQL errors	**not written by MySQL** — on an error no file is created; the error text (number and message) appears only in phpMyAdmin's response	recorded verbatim in RUN_LOG-2.txt (below); the absence of the file is itself the record
Diagnostics (Notes, Warnings)	**not written** — phpMyAdmin runs SHOW WARNINGS inside the same request and shows them; a later SHOW WARNINGS from a new request returns nothing	recorded verbatim in RUN_LOG-2.txt, with the statement's step and time
Timings	not written	"Query took" from the response, in RUN_LOG-2.txt
Timeout counters	yes — H1 is itself captured with INTO OUTFILE before and after each capped statement (files 25, 27, 29, 31)	the four files; the expected movement (+1, 0, 0 per capped statement) is checked from them
The raw phpMyAdmin response	**no**	only method C3 keeps it; under C1 the response is read on screen and transcribed

RUN_LOG-2.txt. One line per step: step, statement id, SHA-256 sent, sent and received time (UTC, from the page clock), HTTP status, phpMyAdmin's message verbatim (rows affected, error, Notes, Query took), file name, file line count, file SHA-256. Two ways to write it inside the folder, your choice (section 11, approval group B): **L1** — I write each line through the shell on your computer at the moment of the step (the shell runs on your

computer; nothing is transferred, but the text originates from this session); **L2** — you paste phpMyAdmin's message into a text file in followup-run\ after each step, and I verify its SHA-256. Under L1 you can check every line against the screen.

Checksums on your computer. After each file is written, its SHA-256 is computed on your computer and appended to SHA256SUMS-2 in the folder root; the 15 October SHA256SUMS stays unchanged. Under C1 the chain is: statement text SHA-256 (approved) → file written by MySQL → file SHA-256 on your computer; no hash is computed anywhere else.

Fallback C2 (no prerequisites, no writes by me): you run the plain Revision 1 text, use phpMyAdmin's Export of the result, save it into followup-run, and I verify its SHA-256 and row count. **C3** (the 15 October route) only with your express approval of the transit.

4. BK1 attribution rules: table plus journal id, reused ids, what never counts as attribution

The identity of a journal leg is the triple (ledger_table, journal_id, Dt). M2 shows that ID is a non-unique index in both ledger tables and that the two tables are separate id spaces, so a bare id names nothing: `account_runing` 26061 and `account_runing_2024` 26061 would be two unrelated rows. Dt (the entry timestamp) is the tie-breaker within a table. **[Observed: M2, G3]**

A journal is the set of rows in one table that share ID and Dt. G3 showed every journal under the 38 numbers has exactly two legs with one Dt; that is the pattern a journal is expected to have, not a rule the schema enforces.

Reused ids. If BK1 returns rows in one table with the same ID but different Dt values, they are treated as different journals that reused an id, each attributed on its own; the case is listed as "id reused" with both Dt values, and nothing is merged. If two rows share ID and Dt but their Cashier differs, they are listed as "id and time shared, cashier differs" and treated as separate until Ravi says otherwise. **[Proposal]**

Attribution to voucher 7134, in four levels, recorded per row:

Level	Rule	Result
A — attributed by number	receipt_no = 7134 **and** the leg is Dr or Cr 298,375.00 (or 596,750.00) **and** description type is "voucher payment" or "bank"	belongs to 7134 (journals 26061 and 26075 are the known cases)

Level	Rule	Result
B — attributed by journal	the other legs of a level-A journal: same ledger_table, same ID, same Dt	belong to 7134
C — candidate, not attributed	receipt_no = 7145 (the voucher's record id, which is **also a voucher number of its own**, so a 7145 row may be another voucher's posting); or a leg on 1-6-8-9750 / 1-6-8-10380 for exactly 298,375.00 or 596,750.00 dated 10–13 July 2026 under any other number; or a 7134 row of another description type	reported with date, Dt, login id and description type as a question for Ravi; never counted
D — same amount, not linked	any other row BK1 returns (same figure, another date or number)	listed so nothing is dropped; stated as unrelated unless Ravi links it from bank records

What never establishes attribution on its own: a matching amount; a matching date; a matching id (7134 or 7145) without the amount and description-type conditions; a matching login id. Two of these together are still level C. **[Proposal]**

How BK1's output is read. The file lists ledger_table first, then journal_id, Dt, L_Code, Dr, Cr, Date, description type, the Invoice_Num marker, aflag and login ids; the ORDER BY groups a journal's legs together (table, date, id). The classification is written into the Ravi pack (section 10) row by row, with the level and the reason.

5. The minimum remaining checks, in order: exact SQL, checksums, capture, stop rules

Ten statement executions, steps 22–31, continuing the 15 October numbering; all read-only on localhost schema avenir (S1, S2, M3 read server variables or information_schema, as H0, H1, M1 and M2 did with your express permission). Every SHA-256 is over the exact text shown, no trailing newline. The plain Revision 1 texts for M3, BK1 and R20.2r (checksums 1687e116…, 74e1b6fa…, f5d973b6…) are in the consolidated plan, section 4; the capture texts below are those texts with the final semicolon replaced by the INTO OUTFILE line. The ten texts are also staged as files statements\capture-c1\NN_ID.sql for the evidence folder (not yet written to your computer).

Common rules. One statement at a time; the SHA-256 gate before sending; H1 immediately before and after each capped statement (expected set +1, set_failed +0, exceeded +0; H1's own run does not count); the phpMyAdmin message transcribed to RUN_LOG-2 before the next step; the file's SHA-256 computed on your computer before the next step. **Stop, no retry, no rewording:** any error (including 3024 time-limit and 1086 file exists), an HTTP status other than 200, a sign-in prompt, a rows-affected count that differs from the file's line count, a checksum difference, a counter movement other than expected, or a missing file after a success message. A syntax refusal of the INTO OUTFILE position (MySQL 8.4 accepts it at the end of a SELECT and of a UNION; if it does not, that is a stop and the text is revised).

Step	Id	Purpose	Hint	Capture	Expected	SHA-256
22	S1	secure_file_priv and version	none	screen → RUN_LOG-2 (one row, no file)	'' or a path covering the folder; 8.4.7	449e8018830b0cce379f0222557d9256fe270064a506a3d06aa612278b06beec
23	S2	FILE privilege of 'root'@'localhost'	none	screen → RUN_LOG-2	1 row: FILE	533f539ff1191a52e92dccff159521059713345238ad2a492eeb7f6d885faa37
24	S3	probe write into followup-run\	none	file 24_S3_probe.tsv	1 row affected; file content `1`	b42a22056474d16623e628b024216a3eee0c8dd73d70d278d2a3c1e7f49e2e16
25	H1	counter baseline	none	file 25_H1.tsv	3 rows; set = 9 unless another timed SELECT ran (then stop and report)	98bbd49db0162e9013745866c4c3daec3d89265e28a3fc72dbb5e7009874109f
26	M3	the three columns on both ledger tables	60 s	file 26_M3.tsv	6 rows, equal types per column	2a0328bc90cad081dd591cef5cdd14a93661b815c7d5ce06c4f6d49e64e0265a
27	H1	counters after M3	none	file 27_H1.tsv	set +1	d8f681d41331300f3b0563ad6ea56deac68c1f98367f450a7393d622db9cf184
28	BK1	bank-side rows for 7134 (section 4 rules)	60 s	file 28_BK1.tsv	≥ 4 rows (journals 26061, 26075)	112d9972e989a65cb136e008262bf8e04b68e9b8de8f808a29ed4148403f7a31
29	H1	counters after BK1	none	file 29_H1.tsv	set +1	093fc07725de254d707a580a9b968acdc6663b2f063a01ba58d9f75ab121d25a
30	R20.2r	candidates by month for the 3,921 (plan 4a checks: monthly totals = R20.1 differences; only/both columns add up)	120 s	file 30_R20.2r.tsv	21 months with not_found > 0, total 3,921	68980153e98d3ddc425d1038661b4d2ca6e19881b82bc0f83e2f75438be7026e
31	H1	counters after R20.2r; new baseline	none	file 31_H1.tsv	set +1 (or exceeded +1 with error 3024: stop)	a9a32ef379741961c7c1ecca66a8005ab6804ddc5eef4983936bf876a4db4803

If S1 or S2 fails, steps 24–31 are not run under C1; the same statements run under C2 with the plain Revision 1 texts (M3 1687e116…, BK1 74e1b6fa…, R20.2r f5d973b6…, H1 b5d71484…), you export each result into followup-run, and I verify.

S1

```
SELECT @@GLOBAL.secure_file_priv AS secure_file_priv, @@version AS
mysql_version;
```

S2

```
SELECT GRANTEE, PRIVILEGE_TYPE, IS_GRANTABLE
FROM information_schema.USER_PRIVILEGES
WHERE GRANTEE = '''root''@''localhost''' AND PRIVILEGE_TYPE = 'FILE';
```

S3

```
SELECT 1 AS probe
INTO OUTFILE 'C:/wamp64/www/avenir/writable/AvenirEvidence/2026-10-14-
stage1/followup-run/24_S3_probe.tsv';
```

H1, step 25 (steps 27, 29 and 31 are identical except for the file name 27_H1.tsv, 29_H1.tsv, 31_H1.tsv; their checksums are in the table)

```
SELECT VARIABLE_NAME, VARIABLE_VALUE
FROM performance_schema.global_status
WHERE VARIABLE_NAME IN ('Max_execution_time_set',
'Max_execution_time_set_failed', 'Max_execution_time_exceeded')
ORDER BY VARIABLE_NAME
INTO OUTFILE 'C:/wamp64/www/avenir/writable/AvenirEvidence/2026-10-14-
stage1/followup-run/25_H1.tsv';
```

M3, step 26

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ c.TABLE_NAME, c.COLUMN_NAME,
c.DATA_TYPE, c.COLUMN_TYPE, c.IS_NULLABLE, c.COLUMN_DEFAULT
FROM information_schema.COLUMNS c
WHERE c.TABLE_SCHEMA = 'avenir'
  AND c.TABLE_NAME IN ('account_runing', 'account_runing_2024')
  AND c.COLUMN_NAME IN ('Dt', 'Cashier', 'Invoice_Num')
ORDER BY c.TABLE_NAME, c.COLUMN_NAME
INTO OUTFILE 'C:/wamp64/www/avenir/writable/AvenirEvidence/2026-10-14-
stage1/followup-run/26_M3.tsv';
```

BK1, step 28

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ 'account_runing' AS ledger_table,
a.ID AS journal_id, a.Recpt_No AS receipt_no, a.L_Code, a.Dr, a.Cr, a.Date,
a.Dt AS entered_at,
       CASE WHEN UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
payment'
            WHEN UPPER(a.Discription) LIKE 'BANK%' THEN 'bank (text
```

```
withheld)'
            WHEN a.Discription IS NULL OR a.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END AS description_type,
       CASE WHEN a.Invoice_Num = '0' THEN '0' ELSE '(withheld)' END AS
line_invoice_marker, a.aflag,
       (SELECT GROUP_CONCAT(u.id ORDER BY u.id) FROM avenir.user_acc_tbl u
WHERE u.user = a.Cashier) AS entering_login_ids
FROM avenir.account_runing a
WHERE a.Recpt_No IN (7134, 7145)
   OR (a.L_Code IN ('1-6-8-9750', '1-6-8-10380') AND (a.Dr IN (298375.00,
596750.00) OR a.Cr IN (298375.00, 596750.00)))
UNION ALL
SELECT 'account_runing_2024', b.ID, b.Recpt_No, b.L_Code, b.Dr, b.Cr, b.Date,
b.Dt,
       CASE WHEN UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
payment'
            WHEN UPPER(b.Discription) LIKE 'BANK%' THEN 'bank (text
withheld)'
            WHEN b.Discription IS NULL OR b.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END,
       CASE WHEN b.Invoice_Num = '0' THEN '0' ELSE '(withheld)' END, b.aflag,
       (SELECT GROUP_CONCAT(u.id ORDER BY u.id) FROM avenir.user_acc_tbl u
WHERE u.user = b.Cashier)
FROM avenir.account_runing_2024 b
WHERE b.Recpt_No IN (7134, 7145)
   OR (b.L_Code IN ('1-6-8-9750', '1-6-8-10380') AND (b.Dr IN (298375.00,
596750.00) OR b.Cr IN (298375.00, 596750.00)))
ORDER BY 1, 7, 2, 5 DESC, 4
INTO OUTFILE 'C:/wamp64/www/avenir/writable/AvenirEvidence/2026-10-14-
stage1/followup-run/28_BK1.tsv';
```

R20.2r, step 30

```
SELECT /*+ MAX_EXECUTION_TIME(120000) */
       DATE_FORMAT(v.W_Date, '%Y-%m') AS voucher_month,
       COUNT(*) AS not_found_by_number,
       SUM(cur.d IS NOT NULL OR old.d IS NOT NULL) AS
candidate_same_date_and_amount,
       SUM(cur.d IS NOT NULL AND old.d IS NULL) AS
candidate_current_table_only,
       SUM(cur.d IS NULL AND old.d IS NOT NULL) AS candidate_2024_table_only,
       SUM(cur.d IS NOT NULL AND old.d IS NOT NULL) AS candidate_both_tables,
       SUM(v.Amount = 0) AS zero_amount_vouchers,
       SUM(v.Amount = 0 AND (cur.d IS NOT NULL OR old.d IS NOT NULL)) AS
zero_amount_candidates
FROM avenir.voucher v
```

```
LEFT JOIN (SELECT a.Date AS d, a.Dr AS amt FROM avenir.account_runing a WHERE
UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%'
           UNION
           SELECT a.Date, a.Cr FROM avenir.account_runing a WHERE
UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%') cur
       ON cur.d = DATE(v.W_Date) AND cur.amt = v.Amount
LEFT JOIN (SELECT b.Date AS d, b.Dr AS amt FROM avenir.account_runing_2024 b
WHERE UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%'
           UNION
           SELECT b.Date, b.Cr FROM avenir.account_runing_2024 b WHERE
UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%') old
       ON old.d = DATE(v.W_Date) AND old.amt = v.Amount
WHERE v.Aflag = 1
  AND NOT EXISTS (SELECT 1 FROM avenir.account_runing a WHERE a.Recpt_No =
v.Vnum AND UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%')
  AND NOT EXISTS (SELECT 1 FROM avenir.account_runing_2024 b WHERE b.Recpt_No
= v.Vnum AND UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%')
GROUP BY voucher_month
ORDER BY voucher_month
INTO OUTFILE 'C:/wamp64/www/avenir/writable/AvenirEvidence/2026-10-14-
stage1/followup-run/30_R20.2r.tsv';
```

6. Recommendation on the two E1 Notes, with its basis

Recommendation. Treat the two Notes as **not** failing Revision 1′s condition 1, and approve R20.2r on that basis (D1 = yes). **[Proposal]**

Basis, in four points.

1. **What they are.** MySQL diagnostics have three levels: Note, Warning, Error. phpMyAdmin printed “Note: #1276 Field or reference 'avenir.v.Vnum' of SELECT #8 was resolved in SELECT #1” and the same for SELECT #9. Both are level Note. **[Observed: E1 response, followup-run\20_E1_response.json]**
2. **What #1276 means.** It is the server′s report that a column used inside a subquery (SELECT #8 and #9 are the two NOT EXISTS subqueries) was resolved against the outer query (SELECT #1). That is the definition of a correlated subquery, which is how the two NOT EXISTS clauses are written on purpose; the Note states that the server read them as intended. It reports no fault, no truncation, no implicit conversion and no change of plan. **[Observed: the message text; Deduced: its meaning]**
3. **It is consistent with condition 4, not against it.** Condition 4 requires each NOT EXISTS to be “a dependent lookup that uses an index on Recpt_No” or an antijoin; the plan shows exactly that (Index lookup on a/b using Recpt_No (Recpt_No = v.Vnum)). A

correlated lookup on v.Vnum is what produces #1276. Reading condition 1 to exclude it would make conditions 1 and 4 contradict each other. **[Deduced from the E1 plan]**

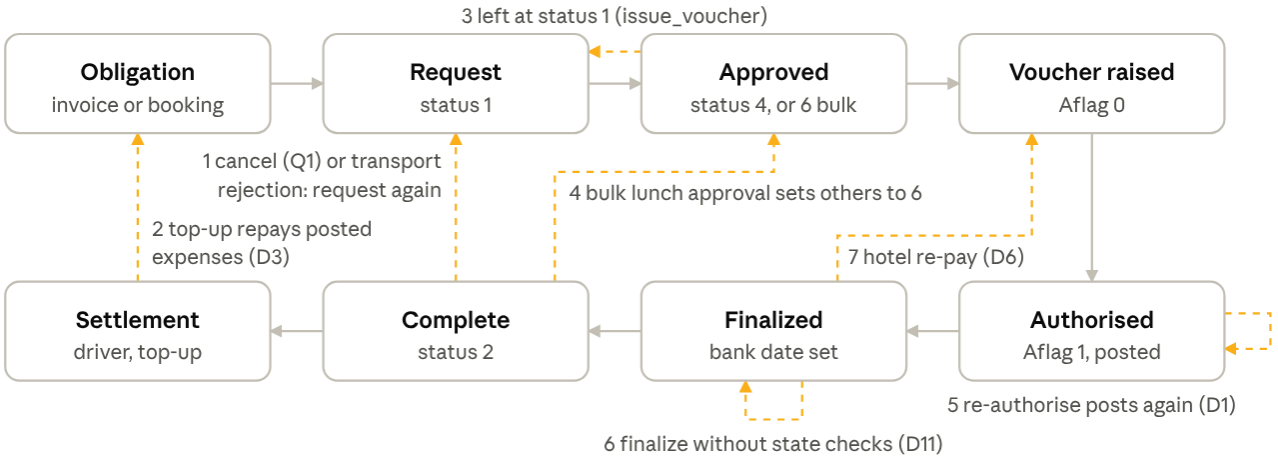
4. **Condition 1's purpose.** Revision 1 wrote "no error and no warning" to catch a plan that MySQL could not build as written: a syntax problem, an ignored hint, a type conversion that defeats an index. None of those is present: the hint was applied (H1 moved after E1), the indexes are used, and the Notes concern resolution, not execution.

What I am not claiming. Not that Notes never matter: a Note about an ignored hint or an implicit cast would be a stop. Not that R20.2r's runtime is known; it is untested, which is why the 120-second cap, the H1 pair and the 3024 stop rule stay. If you prefer the strict reading of condition 1, R20.2r does not run and the 3,921-voucher question stays open; nothing else in this package depends on it.

7. The reported recurring-payment problem: flow map, where a paid item can return, the four kinds of duplicate

Basis. Everything here comes from the code findings already on record (follow-up report section 6 and its route inventory; A2 section 4a; R0 R14–R16; r3.5 sections 12b–12d). No new code was read for this package. Each point is marked with its source and whether its effect is observed in code or deduced. The live system's screens were not examined, so what you saw there is reported, not explained. **[Observed: reports; Deduced where marked]**

A paid obligation can re-enter the payment flow at seven points



Dashboards show paid from tour flags, not from status: 266 at status 4 show flag 1, 21 at status 2 show flag 0 (R0)
The bank transfer is made outside the system; no step here moves money. Numbers 1 to 7 are explained in the table below.

payment flow · 8 states, 7 return points

Solid arrows are the intended flow; dashed arrows are the return points the code allows. Which of them produced what you saw is not established.

7a. The flow and its state markers

State	Marker in the data	Written by (routes)
Obligation	a booking night, a lunch activity, a driver allocation, a supplier bill	booking confirmation, itinerary, allocation screens
Request (status 1)	payment_request.status; the tour's request markers (payment_request, hotel_advance, transport_advance)	request pages; advanceRequestSubmit; tour_transport (two routes)
Approved (4, or 6 for bulk-type)	payment_request.status	P1, P2 (approver), P3 (desk), P4 create_bulk_voucher, P5 save_payment (no status check)
Voucher raised (Aflag 0)	voucher row, v_inv lines; the number from the unlocked counter	P4, P7 lunch_pay, desk issue_voucher, manual vouchers, creditor payments, settlements, hotel_pay, utility bills
Authorised (Aflag 1, ledger posted)	voucher.Aflag; account_runing journal (Dr expense / Cr 1-6-8-9750)	V1 voucher_auth/update; V2/V3 desk Paid; V4 hotel_voucher_accept; V5 transfer_amount (creates already authorised)
Finalized (bank date)	voucher.finalized, bank_date; the desk's own Paid record (tpd_bank_approval)	Bank Payment Check mark_finalized; desk Paid
Complete (status 2)	payment_request.status	P6 payment_request_final, P7 lunch_pay (no status check)
Settlement	driver settlement voucher; top-up	DriverPaymentSettlementController (two identical routes); accept_approved_expenses

7b. Where a paid or completed obligation can come back

#	Return point	Mechanism	Source
1	Complete → Request	Q1 cancel_payment_request has no status check: an approved, vouchered or completed request can be cancelled (status 3, tour marker reset) and requested again. Rejecting a Transport Advance request resets every request row of that driver on the tour, paid ones included, so they can be requested again	A2 Q1 [:1536–1556]; follow-up report F-2/F-3 [Observed code; effect Deduced]
2	Settlement → Obligation	accepting expenses for an already settled driver raises a top-up equal to all posted expenses, paid ones included	D3, TourAccountsController :2388–2435 [Observed; effect Deduced]
3	Approved → Request	the desk's issue_voucher switches a bulk-type request from 6 to 1 around its internal call and back; a failure in between leaves it at 1, where the request queue shows it	A2 P4 note; security proposal (Observed)
4	Complete → Approved	approving a bulk-type lunch request sets every other request for the same hotel account to 6, whatever its status; bulk rejection sets a bulk-type request to 6 (approved) instead of 5	r3.5 12b; R0 R14, F7 [Observed]
5	Authorised → Authorised	V1 refuses only a voucher marked rej, so a second authorisation posts the ledger again (voucher 7134 is the instance found)	D1 [Observed code; 7134 Observed data]

#	Return point	Mechanism	Source
6	Finalized → Finalized	mark_finalized checks neither Aflag nor finalized: it can finalise an unposted or cancelled voucher or overwrite a bank date, so a voucher can look newly payable or newly paid	D11 [Observed code]
7	Finalized → Voucher raised	nights paid through the desk keep a paid-dollar count the old hotel_payment and hotel_pay routes compare against, so they can be paid again	D6, R2 [Observed code; effect Deduced]
—	Display only	dashboards and the Payment Requested report derive “paid” from tour flags, not from stored status; the flags disagree with status today (266 at status 4 show flag 1, 21 at status 2 show flag 0)	R0 R16; r3.3 5b [Observed data]

Also on record, feeding duplicates rather than returns: no creation route has a duplicate check, so a double submit or a bill accepted and then paid yields two vouchers (D8); two writers can take one voucher number, and lines of a failed voucher attach to the other (D9) — on 15 October M2 showed a unique index on voucher.Vnum, which stops two voucher rows sharing a number but not the v_inv lines; V4 posts a voucher with lines on k tours k times (D2); two simultaneous accepts post every expense twice (D5); settlement charges the transport cost twice (D7). **[Observed code; effects Deduced]**

7c. Four different things, kept apart

Kind	What it is	How it shows	How it is told apart in the data
Display duplication	one record shown more than once, or a paid record shown as unpaid, because the screen's query joins or filters wrongly or reads flags instead of status	the same request or invoice twice on a panel; a paid item back in the list	one row in payment_request / voucher / v_inv per obligation; the screen's query explains the repeat
Duplicate records	two requests, two vouchers or two lines for one obligation (return points 1–4, D8, D9)	two PV numbers for one bill; a request re-raised after completion	two rows with different ids for one obligation; each must stay, labelled
Duplicate postings	one voucher posted twice to the ledger (return point 5; D2, D4, D5, D7)	expense and payable overstated; the 298,375.00 case	two journals under one voucher number; the bank side may be single
Duplicate bank payment	money sent twice	only on the bank statement	not visible in this system: the application records payments, it does not make them; a second transfer needs a person acting on a duplicate record or a reappeared item

The LKR 298,375.00 finding is the third kind. Whether it is also the fourth is Ravi's question against the statement (section 10).

7d. The one blocker that needs a word from you, not an investigation

Your two observations name a "payment panel" and "invoices". The materials on record describe several screens that list payments (the Payment Requested report and its new version, the Payment Approval page, the Tour Payment Dashboard, the Tour Payment Desk, Bank Payment Check, Supplier Payables) and two things called invoices (v_inv voucher lines; creditor invoices). Which screen and which column you saw decides which

query is read next. **Needed from you:** the screen's name or address as you see it on the live system, the column that repeats, and one example reference (a tour number or PV number) if you have it. With that, one static read of that screen's controller on localhost, under approval, would say whether the repeat is kind 1, 2 or 3. Without it, the next step would be a guess across six screens, which is the investigation expansion you asked me not to make. **[Blocker]**

8. Focused test cases (not executed): environment and approvals they need

None of these has been run, and none runs under this instruction. Each is a write test: it creates TEST requests, vouchers and journals and exercises a route twice or in parallel. They are specified so that, once the Stage A guards exist, each guard has a test that fails on today's code and passes after. The expected results on today's code are deduced from the code findings, not observed. **[Proposal]**

#	Case	Set-up (TEST records only)	Action	Today's code is expected to	A correct system must
RT-1	Repeated authorisation	one TEST voucher at Aflag 0	V1 twice: a double click, then a second tab	post two journals (D1; the 7134 pattern)	post once; refuse the second with a message; no second journal
RT-2	Simultaneous approvals	one TEST request at status 1	P1 and P2 at the same moment from two logins	both succeed; one request, two outcomes, two vouchers possible (D5-like)	exactly one approval; the other refused on re-read under the lock
RT-3	Simultaneous expense accepts	TEST driver with two posted expenses	accept_approved_expenses twice in parallel	every expense posted twice; two vouchers (D5)	once; one voucher
RT-4	Partial payment	TEST voucher for 100,000; TEST payment of 60,000 recorded	record the balance	legacy routes: no allocation model; the desk's allocations exist only in the r3.5 design	the balance pays once; the request never completes early; totals equal the obligation

#	Case	Set-up (TEST records only)	Action	Today's code is expected to	A correct system must
RT-5	Interrupted processing, desk Paid	TEST voucher; a failure injected after finalized is set	retry Paid	retry refused; exchange difference and bank charge left unposted (F-4)	either complete the remaining postings or roll the step back; never a half-posted Paid
RT-6	Interrupted processing, issue_voucher	TEST bulk-type request at 6	failure injected inside issue_voucher's internal call	request left at 1, back in the queue (return point 3)	status restored to 6; nothing re-queued
RT-7	Paid item returns: cancel	TEST request at status 2 with a finalized voucher	Q1 cancel_payment_request, then request again	cancelled to 3 and re-requested; a second request for a paid obligation (return point 1)	cancel refused unless status 1 and no voucher
RT-8	Paid item returns: transport rejection	TEST driver with one paid and one requested advance on a tour	reject the requested one	both rows reset; the paid one can be requested again	only the rejected row changes
RT-9	Paid item returns: bulk lunch approval	two TEST requests for one hotel account, one at 2	approve the other as bulk-type	the completed one set to 6 (return point 4)	the other request untouched
RT-10	Bulk rejection of a bulk-type request	TEST bulk-type request at 1	bulk reject	status 6, appears approved (R0 F7)	status 5
RT-11	Finalize without state	TEST cancelled voucher; TEST voucher at Aflag 0	mark_finalized on each	both accepted; bank date written (D11)	both refused
RT-12	Hotel re-pay after desk Paid	TEST night paid through the desk	hotel_payment and hotel_pay	paid again (D6)	refused: night already paid

#	Case	Set-up (TEST records only)	Action	Today's code is expected to	A correct system must
RT-13	Top-up after settlement	TEST driver settled; one expense accepted after	accept	top-up equal to all posted expenses, paid included (D3)	top-up equal to the unpaid balance only, or refused
RT-14	Duplicate creation	one TEST utility bill	accept, then pay; and a double submit on a creation form	two vouchers (D8)	one voucher; the second refused by a business key
RT-15	Number race	two creations started together	write lines, then vouchers	lines of a failed voucher attach to the other's number (D9); the unique index on Vnum now refuses the second voucher row	lines tied to their own voucher only; a refused voucher leaves no lines
RT-16	Display from state	the R0 pattern: a TEST request at 4 with flag 1; one at 2 with flag 0	open each panel	panels show the flag, not the status	panels show the stored status

Environment they would require. An isolated copy restored from a verified development backup, with TEST logins, groups and records only; every run starting from a dump and ending with the copy restored from it (the NC.1 rule of r3.5); the exact-database guard in every test; the ledger hashed before and after each case. Which copy: avenir_rv (C:\wamp64\www\avenir_rv, its own database on the same MySQL server) is the only candidate inside C:\wamp64\www, and whether it is within the approved structure is open (register U-6); the cloud test copy is excluded by the restriction. The verified restore that precedes it needs the restore-proof route (register U-5). Development is never used for write tests; live is not covered.

Approvals each run would need, all separate from this package: the copy and its refresh from a backup (T2 or your alternative); the TEST fixture (named records, named logins); each test set by its number; the guards under test, built as Stage A items, for the "must" column to be testable. Without the guards, the tests can only confirm today's behaviour, which you already treat as unverified; running them for that alone is not recommended.

9. Repair-and-test proposal, prioritised: software faults apart from historical-data reconciliation

Two streams that must not be mixed. Stream S fixes the code so that the return points and duplicate routes close; it changes no historical record. Stream H reconciles the records as they stand, with Ravi, and decides corrections; it changes no code. A code fix never "cleans" data, and a data correction never relies on untested code. The April 1 rebuild, when its year is confirmed, belongs to stream H and starts only after stream S has closed the routes that would otherwise re-create the same faults in the new records. **[Proposal]**

Stream S — software faults, in priority order

Priority	Item	Closes	Where (follow-up report section 7 / register)	Test
S-1	Authorisation guard: V1 refuses a voucher not at Aflag 0, under the desk's voucher lock; V4 refused; V5 requires an authorised original and a checked amount	return point 5; D1, D2, D10	A1, A2, A6	RT-1
S-2	Status guards with compare-and-set and a per-request lock on P1, P2, P4, P5, P6, P7, Q1: approval and rejection only from status 1; completion only from 4 or 6 with a finalized voucher; cancellation only from 1 without a voucher; bulk rejection to 5	return points 1, 3, 4; RT-2, RT-7, RT-10	A3, A4 (C0.8)	RT-2, RT-6, RT-7, RT-9, RT-10
S-3	Transport-advance rejection changes only the rejected row; bulk lunch approval changes no other request	return points 1 and 4	A5	RT-8, RT-9

Priority	Item	Closes	Where (follow-up report section 7 / register)	Test
S-4	mark_finalized refuses an unposted, cancelled or already finalized voucher and never overwrites a bank date	return point 6; D11	A7	RT-11
S-5	Hotel re-pay routes refuse a night the desk has paid (or are refused outright, as the register recommends for R2)	return point 7; D6	A8; register R2	RT-12
S-6	Expense acceptance: a lock, one posting per expense, top-up limited to the unpaid balance; approve_expense unrouted	return point 2; D3, D4, D5	A9	RT-3, RT-13
S-7	Business keys and the counter: one voucher per bill or request (unique key), lines tied to their voucher in one write; the unique index on Vnum already present	D8, D9	A10 part; r3.5 section 14	RT-14, RT-15
S-8	Panels read stored status, not tour flags; flags become display-only or are retired	the display kind of duplicate	r3.3 5b	RT-16
S-9	CSRF check on payment routes; voucher_print escaping	not a duplicate cause; closes the cross-site path	A10	existing E8 tests

Order of work for each item: build on the isolated copy, run its RT test and the regression set, package with before-copies and checksums, your approval, then development only after a verified backup; live is not covered. Items S-1 to S-4 are the containment in refined form (register and follow-up report section 9); approving them settles containment, not A0 or D14.

Stream H — historical-data reconciliation, with Ravi

Priority	Item	Evidence on record	Decision needed from
H-1	Voucher 7134: the duplicate ledger posting (journals 26061, 26075, both 13 July 2026); by your clarification of 15 October no duplicate bank payment was found in the monthly bank checks (section 12)	G1c, G2r, G3, G4; BK1 when run	Ravi: which journal to reverse; bank statement check
H-2	The 36 finalized sup = 0 bank transfers with no credit to 1-6-8-10380	Stage One section 7; R0	Ravi: paid on the statement? one correcting method for all?
H-3	The eight unbalanced journals and the fifteen future-dated journals, including the two double-looking pairs	R21, R22, L2	Ravi: correct, reverse or annotate; nothing deleted
H-4	The 3,921 authorised vouchers with no ledger row under their number; how many have a same-date, same-amount candidate	R20.1; R20.2r when run	Ravi: sampling plan; which are old-ledger postings under other numbers
H-5	Request-status inconsistencies: 266 at status 4 with flag 1, 21 at status 2 with flag 0, 15 at status 6	R0	you and Ravi: which records are paid in fact
H-6	The three rejected hotel requests whose accruals were deleted	R0 R14	Ravi: restore from backup evidence or annotate
H-7	Suspected duplicate records from the return points (requests re-raised after completion; two vouchers per bill)	not yet listed; a read-only listing would be a new statement set for approval	you: whether to list them before the rebuild

Priority	Item	Evidence on record	Decision needed from
H-8	The April 1 rebuild: scope, year, opening balances	not started; no date assumed	you and Ravi, after H-1 to H-7

Stream H changes nothing until Ravi and you decide each item; every suspected duplicate stays in the data, labelled (section 10). Stream S changes no historical record at all.

10. Accounting evidence preserved: suspected duplicates kept identifiable; Ravi's pack and questions (not sent)

Rule applied to everything in this package. No historical record is deleted, merged, reversed or cleaned up by me, and none is proposed to be until Ravi and you decide. A suspected duplicate is kept in the data and listed with its references (ledger table and journal id, voucher number and voucher record id, request id) and the reason it is suspected, so that whoever corrects it can see what it was. The LKR 298,375.00 finding is a **duplicate posting**; it is not a finding of a duplicate bank payment. **[Proposal, applied]**

Suspected-duplicate register, as it stands (to be extended by BK1 and R20.2r):

Reference	What is suspected	Reason	Status
account_runing journals 26061 and 26075, voucher 7134 (record id 7145)	one of the two is a duplicate posting	identical legs, 64 minutes apart, two logins; every other sup = 0 voucher has one journal	kept; which one is Ravi's decision
account_runing journals 25668 and 25669 (2525-04-16, 25,000.00 each)	double-looking pair	same date, accounts and amount, consecutive ids	kept; R22
account_runing journals 28694 and 28695 (2027-01-04, 123,310.00 each)	double-looking pair	as above	kept; R22
the 36 finalized sup = 0 transfers	payable never cleared to the bank ledger	no credit to 1-6-8-10380 found	kept; H-2

Ravi's evidence pack (prepared, not sent; sent only on your word), all from the evidence folder, bank account numbers withheld:

1. R23.1's row for 7134: voucher number, record id 7145, date 2026-07-10, Bank Transfer, bank ledger code 1-6-8-10380, 298,375.00, finalized, bank date 2026-07-13 (stage1-results\R23.1.txt).
2. G2r and G1c rows for 7134 (followup-run\10, 08): amount 298,375.00; credits to 1-6-8-9750 596,750.00 in 2 journals; no debit to 9750; no 2024-table rows.
3. The four G3 legs (followup-run\12): journal 26061 — Dr 6-9-23-11013 / Cr 1-6-8-9750, entered 2026-07-13 15:24:03, login id 75; journal 26075 — the same, entered 16:28:48, login id 1; both dated 2026-07-10, aflag 3, no Bank_Date.
4. G4 line 7204 (followup-run\14): account 6-9-23-11013, 298,375.00, 2026-07-10.
5. BK1's rows, each with its attribution level and reason (section 4), once run.
6. Stage One section 7: the 36 sup = 0 transfers naming 1-6-8-10380 with no credit to it.
7. R21, R22 and L2 rows for the eight unbalanced and fifteen future-dated journals, with aflag and Bank_Date (followup-run\18; stage1-results).

Questions for Ravi, against the bank records he holds:

1. Does the Commercial Bank statement show one payment of LKR 298,375.00 for voucher 7134 on or about 10–13 July 2026, or two?
2. Which of journals 26061 and 26075 is to be reversed, with what date, and should the reversal reference both ids?
3. Should 6-9-23-11013 carry one charge of 298,375.00 for this voucher after correction?
4. Was any bank-side journal (Dr 1-6-8-9750 / Cr 1-6-8-10380) ever posted for this voucher, under what number; and if none, how is the payable to be cleared for it and for the other 35?
5. For journals 25668/25669 and 28694/28695: one event or two?
6. For the eight unbalanced journals: correct by a balancing entry, reverse and re-post, or annotate and leave?
7. For the rebuild from April 1: which opening balances he will certify, and from which records.

Nothing is sent to Ravi, no record is corrected, and no staff assignment is made under this package.

11. Outstanding approvals, grouped (D8 optional, non-blocking); blockers that cannot be resolved from existing material

Each approval is for one named act on localhost only. Approving a statement approves one read-only run of that exact text; approving a capture method approves writes of result files into the approved evidence folder only. Nothing here installs, changes data, schema, permissions or staff, or sends anything to anyone.

Group A — the remaining reads (one line can cover the group)

#	Approval	Covers
A-1	S1 (449e8018…) and S2 (533f539f…): the two server reads that decide the capture method	section 3a, 5
A-2	The two E1 Notes do not fail condition 1 (recommendation: yes, section 6)	R20.2r
A-3	M3, BK1 and R20.2r by their plain checksums (1687e116…, 74e1b6fa…, f5d973b6…) **and**, for method C1, by their capture checksums (2a0328bc…, 112d9972…, 68980153…) with the four H1 captures (98bbd49d…, d8f681d4…, 093fc077…, a9a32ef3…)	section 5
A-4	Relabel follow-up report section 10 from B1 to BK1 (text and checksum unchanged)	section 4

Suggested wording: “Group A approved: S1, S2; E1 Notes accepted; M3, BK1, R20.2r and H1 by the checksums in package section 5; relabel to BK1.”

Group B — the writes into the approved folder

#	Approval	Covers
B-1	S3, the one-row probe write by MySQL into followup-run\ (b42a2205…)	section 3a
B-2	Capture method: **C1** (MySQL writes the result files) if S1–S3 pass, else **C2** (your export into the folder); **C3** only if you expressly approve the transit	section 3
B-3	RUN_LOG-2.txt: **L1** (written line by line by the shell on your computer, text from this session) or **L2** (you paste phpMyAdmin's messages; I verify)	section 3b
B-4	SHA256SUMS-2 appended on your computer after each file; the 15 October SHA256SUMS untouched	section 3b
B-5	The six report exports into reports: **W1** (I transfer the six files by their checksums) or **W2** (you save them; I verify); plus the ten capture texts into statements\capture-c1\ by the same route	plan section 6; this section 5

Suggested wording: "Group B approved: S3; C1 with C2 fallback; L1; SHA256SUMS-2; W1 for the six exports and the ten capture texts."

Group C — later, not blocking the next step

#	Item	Status
C-1	Stream S items S-1 to S-9: build on an isolated copy, then development after a verified backup	needs the copy (register U-6) and the restore-proof route (U-5) settled first
C-2	Stream H items H-1 to H-8 with Ravi; when and how he is contacted	yours
C-3	The suspected-duplicate listing (H-7): a new read-only statement set for approval	only if you want it before the rebuild
C-4	The register's open items on backup, release and restore-proof paths and avenir_rv's standing (U-3 to U-6); the text of the structural restriction (U-1)	yours and IT's

Optional, never blocking

D8: whether the readable part of the old R20.2 body (phpMyAdmin's settings form, token removed) is saved into followup-run\ by the method chosen in B-2, or left out. Nothing waits on it.

Blockers that existing material cannot resolve

#	Blocker	What is needed
X-1	The screen and column behind "paid items reappear" and "invoices appear two or three times" (section 7d)	one line from you naming the screen as you see it, the column, and an example reference; then one static read of that screen's query on localhost, under approval
X-2	Whether C1 is possible at all	S1, S2, S3 (group A-1, B-1); until run, the capture method is undecided and C2 is the only route that needs nothing
X-3	Whether Dt, Cashier and Invoice_Num exist on account_runing_2024	M3; BK1 is not run before it
X-4	The isolated copy for any write test or build (section 8)	the restore-proof route and avenir_rv's standing, from you and IT
X-5	The standing structural restriction's own text	from you; this package applies your 12:01 UTC instruction and the present one as written

Recommended next step, once Group A and Group B are answered: you sign in to phpMyAdmin; steps 22–24 (S1, S2, S3) decide the capture method; steps 25–31 run in order with their stop rules; one short results report; then Ravi's pack is completed with BK1's classified rows for your decision on contact. Everything else waits for Group C.

12. Clarification of 15 October: classification of voucher 7134

Your clarification, recorded as given. The monthly bank checks found no duplicate bank payments. **[Owner's statement, 15 October; not verified by me, and not verifiable from the system's data]**

Classification, applied from here on in every document and in Ravi's pack: voucher 7134 carries a **duplicate ledger posting of LKR 298,375.00**: journals 26061 (entered 2026-07-13 15:24:03, login id 75) and 26075 (entered 2026-07-13 16:28:48, login id 1), both dated 2026-07-10, each Dr 6-9-23-11013 298,375.00 / Cr 1-6-8-9750 298,375.00, in account_runing. It is

not described as money paid twice, and nothing in the evidence says it was. The accounting correction is one reversal, of the journal Ravi identifies as the duplicate, confirmed by him against the bank reconciliation; the original records are preserved and neither journal is changed until then. **[Observed: G3; classification per your instruction]**

What this changes in the earlier text. Section 2's open question on the bank side and Stream H item H-1 now carry this classification; question 1 of Ravi's list becomes a confirmation against the reconciliation rather than an open question. BK1 (sections 4 and 5) keeps its purpose: to show whether any bank-side or clearing row exists for 7134 under any number, which the reconciliation will need; it is not evidence about a payment being made twice.

13. Completion note of 15 October and where the next stage is recorded

This report is complete: sections 0–12 are all present in this document (the PDF you had ended at section 8 because it was exported before sections 9–11 were written; this export carries them). Internal references were checked against the headings: section 11's groups refer to sections 3, 4, 5, 6 and 7d; section 12 refers to sections 2, 4, 5 and 10 and to Stream H item H-1; the register items cited (SC-06, SC-08, U-2 to U-7) are in the CodeIgniter 4.0 Structural Compatibility Conflict Register. **[Observed]**

Where the next stage lives. Your instruction of 15 October to prepare and test the authorisation guard on the isolated localhost copy, and your later clarification that the objective is a reliable system for fresh entry with historical reconciliation set aside, are carried out and recorded in a separate document, **Revised Implementation Plan: Avenir Payment Foundations**: the test-copy confirmation and backups, the guard's changes file by file, the test set and its evidence, the rollback, the stage-by-stage repair list, the acceptance criteria, the clean-database preparation and the blockers. Ravi's pack for voucher 7134 (section 10 here, classified per section 12) stays prepared and unsent; correction of existing entries is kept apart from prevention of future duplicates, which the guard provides.

Unchanged by that stage: the live staff system; the development copy at C:\wamp64\www\avenir and its database; every historical record; staff assignments and permissions.